睿尔曼智能 CRM — 自动化开发流水线

> **版本**: v2.2
> **依赖文档**: 04-系统概要设计.md §13 · 07-Sprint任务卡.md · 09-测试机搭建计划.md
> **定位**: 全自动 AI 开发流水线设计 —— 人只写设计文档，机器完成编码→Review→测试→迭代→部署
> **核心理念**: 你描述系统方案 → Orchestrator 读取方案 → 5 台机器自动协作 → 零人工参与的开发循环
> **v2.2 变更**: Orchestrator 从"重型定制系统"重新定位为"薄调度层"——aider 已覆盖编码→测试→修复的核心循环，Orchestrator 只负责跨机器调度+任务编排+结果汇总+通知

1. 流水线总体架构与理念

1.1 范式转换：从「人指挥 Copilot」到「机器自治」

旧模式（v1.0）中，你打开 5 个 VS Code 窗口，逐窗口给 Copilot 下达指令、巡回 Review、手动 push/pull。
你是整个流水线的 **瓶颈**——4 个窗口的产出速度受限于你切换 Review 的速度。

新模式（v2.0）中，你的角色变为 **系统方案设计者**：

```
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
│  旧模式 (v1.0):  你 ──指令──▶ Copilot ──代码──▶ 你 Review ──▶ git push │
│                  ↑                                  │                 │
│                  └──────────── 人工循环 ──────────────┘                │
│                                                                      │
│  新模式 (v2.0):  你 ──设计文档──▶ Orchestrator ──自动──▶ 5台机器协作    │
│                                   ↑          自动编码 → 自动Review      │
│                                   │          → 自动测试 → 自动迭代      │
│                                   │          → 自动部署                 │
│                                   └── 只在异常时通知你 ──               │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

你的唯一职责：编写/更新 `docs/` 和 `contracts/` 中的设计文档与接口契约。

机器的全部职责：读取文档 → 拆解任务 → 编码 → Review → 测试 → 迭代修复 → 部署 → 报告。

1.2 流水线总体架构

```
                    ┌──────────────────────────────┐
                    │   你（方案设计者）              │
                    │   编写 docs/ + contracts/     │
                    │   git push 更新设计文档        │
                    └──────────────────────────────┘
                                │ docs changed event
                                ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                    orchestrator (172.16.14.201)                          │
│                      ┌──────────────────────────┐                        │
│                      │   🎯 Orchestrator         │                        │
│                      │   (薄调度层 ~200行)        │                        │
│                      │                           │                        │
│                      │  ① 解析任务列表            │                        │
│                      │  ② SSH 分发 aider 指令     │                        │
│                      │  ③ 收集 exit code          │                        │
│                      │  ④ 触发云端 Review API     │                        │
│                      │  ⑤ 判定：通过/重试/升级    │                        │
│                      │  ⑥ 推送报告到钉钉          │                        │
│                      └──────────────────────────┘                        │
│                          │      │      │                                 │
└──────────────────────────┼──────┼──────┼─────────────────────────────────┘
                SSH + aider │      │      │ SSH + aider
          ┌─────────────────┘      │      └──────────────────┐
          ▼                        ▼                         ▼
┌────────────────┐   ┌────────────────┐   ┌────────────────┐   ┌────────────────┐
│ 4090           │   │ mac_min_8T     │   │ gateway        │   │ data_center    │
│ (.11.194)      │   │ (.12.50)       │   │ (.14.215)      │   │ (.14.90)       │
│                │   │                │   │                │   │                │
│ Agent 引擎     │   │ CRM 后端+前端  │   │ 网关+钉钉      │   │ PG热备+监控    │
│ + vLLM A       │   │ + DB           │   │                │   │                │
│ (Agent推理)    │   │                │   │                │   │                │
│ aider 编码     │   │ aider 编码     │   │ aider 编码     │   │ aider 编码     │
│ agent/ 目录    │   │ crm/+frontend/ │   │ deploy/nginx/  │   │ deploy/monitor │
└────────────────┘   └────────────────┘   └────────────────┘   └────────────────┘
          │                  │                  │                  │
          └──────────────────┴──────────────────┘                  │
                               │ git push                           │
                               ▼                                    │
                     ┌──────────────────┐  post-receive hook       │
                     │ Git 裸仓库        │──────────────────────────▶│
                     └──────────────────┘
```

```
│ mac_min_8T:        │                          │
│ /opt/git/          │◀── 测试报告 ──────────────│
│ ai-crm.git         │                          │
└────────────────────┘                          │
          │                                     │
          │ webhook / polling                   │
          ▼                                     │
┌────────────────────┐                          │
│ Orchestrator       │◀── 报告结果 ──────────────┘
│ 判定下一步动作      │
│ ├─ 🟢 通过→下个任务│
│ ├─ 🔴 失败→重试    │
│ └─ ⚠️ 超限→通知人 │
└────────────────────┘
```

1.3 五台机器的角色定义

机器	SSH 别名	旧角色 (v1.0)	新角色 (v3.0)
orchestrator	`ssh orchestrator`	— (新增)	**Orchestrator 调度中心** + **pytest 本地执行** + 云端多模型 Judge API 调用 + 云端 API (AutoReview/DocParser) + aider 编码 `orchestrator/` + **预发布验收 (Staging)**（独立 Staging DB/配置）
4090	`ssh 4090`	你开窗口对 Copilot 说话	**vLLM 实例 A (生产 Agent 推理专用)** + aider 自动编码 `agent/`
mac_min_8T	`ssh mac_min_8T`	你开窗口对 Copilot 说话	aider 自动编码 `crm/` + `frontend/` + Git 裸仓库 + DB
gateway	`ssh gateway`	你开窗口对 Copilot 说话	aider 自动编码 `deploy/nginx/` + `deploy/dingtalk/`
data_center	`ssh data_center`	你开窗口对 Copilot 说话	aider 编码 `deploy/monitoring/` + `scripts/` + PG热备+监控

> ~~aliyun-agent（云端 L20, Staging）~~ — 已移除，预发布验收由 orchestrator 承担，vLLM C 取消。

> **测试内迁架构**：编码 → Review → 测试全部在内网完成，反馈延迟从分钟级降为秒级。AutoReview 和 DocParser 走云端 API（强模型更适合代码审查和文档解析），Agent 集成测试推理调用 4090 vLLM A（内网 <1ms）。

1.4 核心工具链

工具	用途	部署位置
aider	AI 编码 CLI，通过 OpenAI 兼容 API 连接 **Claude Opus 4.6**，自动读上下文→生成代码→commit	4090, mac_min_8T, gateway, data_center, orchestrator
vLLM 实例 A	Qwen3-30B-A3B 推理服务，**生产 Agent 推理 + 测试 Agent 推理共用**（开发期无生产流量，测试独占；上线后分时复用）	4090 (2×4090)
~~vLLM 实例 B~~	~~已移除~~ — AutoReview/DocParser 改用云端 API，Agent 测试推理统一调用 vLLM A	~~orchestrator~~
~~vLLM 实例 C~~	~~已移除~~ — aliyun-agent 已从架构中移除，预发布验收由 orchestrator 承担，Agent 推理统一使用 vLLM A	—
Orchestrator	薄调度层脚本（~200 行 shell/Python），解析任务列表→SSH 分发 aider 指令→收集 exit code→判定下一步。**不重复实现 aider 已有能力**（见下方说明）	orchestrator (专用服务器)
pytest	自动化测试，编码完成后本地触发，零网络延迟	orchestrator (本地执行)
云端多模型 Judge	GPT-4.1/Qwen-Max/DeepSeek-V3 三模型交叉评估	orchestrator 发起 API 调用 → 云端
Git + post-receive hook	代码版本管理 + 推送自动触发测试	mac_min_8T (裸仓库), orchestrator (hook 触发本地测试)
钉钉机器人	进度通知、异常告警、每日摘要报告	gateway (发送)

1.5 Claude Opus 4.6 调用配置（实施方式）

编码统一通过一步 API 的 OpenAI 兼容网关调用 Claude Opus 4.6：

```
export AIDER_MODEL="openai/claude-opus-4-6"
export OPENAI_API_BASE="${OPENAI_API_BASE}"   # 见 .env
export OPENAI_API_KEY="${OPENAI_API_KEY}"     # 见 .env
```

> 已完成 Docker 冒烟验证： `aider --model openai/claude-opus-4-6 --message "reply exactly: OK"` 返回成功。

1.6 全自动开发循环（零人工参与）

```
┌──────────────────────────────────────────────────────────────────┐
│                    全自动开发循环（Autonomous Dev Loop）                │
│                                                                  │
│   ┌──────────┐     ┌──────────┐     ┌──────────┐     ┌──────────┐  │
│   │ 1.解析   │──→│ 2.编码   │──→│ 3.Review │──→│ 4.测试   │  │
│   │ 设计文档 │     │ aider×5  │     │ LLM审查  │     │ 本地自动 │  │
│   └──────────┘     └──────────┘     └──────────┘     └──────────┘  │
│        ▲                                                  │       │
│        │          ┌──────────┐     ┌──────────┐           │       │
│        │          │ 6.部署   │◀──│ 5.判定   │◀──────────┘       │
│        │          │ 服务重启 │     │ 通过?    │                   │
│        │          └──────────┘     └──────────┘                   │
│        │                                 │                        │
│        │    ┌─ 🟢 通过 → 取下一个任务 ──┘                          │
│        │    │                                                     │
│        │    ├─ 🔴 失败 → 生成修复指令 → 回到 [2.编码]（最多3次）     │
│        │    │                                                     │
│        └────┤                                                     │
│             └─ ⚠️ 3次仍失败 → 钉钉通知人工介入                       │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘

单个任务周期: 5-15 分钟
每天可完成: 50-100 个独立编码任务
每天自动迭代轮次: 无上限（持续运行）
```

1.6 人工介入的触发条件

流水线设计为**全自动**，但以下情况会通过钉钉通知人工介入：

#	触发条件	通知内容	人工操作
1	编码任务连续 3 次 Review 失败	失败原因 + 相关代码片段	修改设计文档/契约，或手动修复代码
2	测试评分低于基线（多模型加权评分 < 3.0）	失败用例 + Agent 回答对比 + 各Judge评分明细	调整 Prompt 设计或业务逻辑
3	硬件故障（SSH 不通、GPU 异常）	故障机器 + 错误日志	物理排查/重启
4	Git 冲突（串行提交下极低概率）	冲突文件列表	手动解决冲突
5	Sprint 里程碑检查点	Sprint 完成度报告	确认是否进入下一 Sprint
6	设计文档与代码实现偏差 > 30%	偏差分析报告	更新设计文档或回退代码

> **原则**：通知是为了让你**知道**并**决策**，不是让你**操作**。
> 大部分情况下你只需在钉钉回复「继续」或「暂停」。

2. Orchestrator 核心设计

> **关键认知：aider 已有能力 vs Orchestrator 职责边界**
>
> aider 本身已是一个完整的自动编码循环：读取上下文文件 → LLM 生成代码 → 自动写入文件 → git commit → 运行 lint/test → 测试失败自动生成修复指令 → 重试直到通过或达到上限。因此 **Orchestrator 不需要从零实现编码、Review、测试、修复等能力**，只需要做 aider 做不到的事：**跨机器调度 + 任务编排 + 结果汇总 + 通知**。
>
> 实际实现路径：
>
> - **S1 Day 1-2**：编写 `orchestrator.sh` （~200 行），能 SSH 到 5 台机器运行 aider → 自动化立即可用

> - **S3-4 渐进增强**：加入 DocParser（云端 API 一次调用）、钉钉通知、多模型 Judge
> - **S5+**：按需加入 Redis 任务队列、并发调度等高级功能
>
> 以下伪代码展示**逻辑架构**（各模块的职责和数据流），实际实现会比伪代码简单得多，因为 aider 已覆盖编码→测试→修复的核心循环。

Orchestrator 是整条流水线的调度层，运行在专用服务器 orchestrator (172.16.14.201, Xeon 64C + 128G + 2×4090)，以 Python/Shell 脚本方式运行。AutoReview 和 DocParser 走云端 API（强模型更适合代码审查和文档解析），Agent 集成测试推理统一调用 4090 上的 vLLM A（内网延迟 <1ms，测试环境与生产推理配置完全一致）。orchestrator 的 2×4090 GPU 释放给其他计算任务。pytest 在本机执行，编码→测试反馈全内网闭环。

2.1 Orchestrator 架构

```
                    Orchestrator（专用服务器）
┌──────────────────────────────────────────────────────────────────┐
│                                                                  │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐ │
│  │ Doc Parser   │  │ Task Engine  │  │ Dispatcher   │  │ Judge        │ │
│  │ 文档解析器   │→│ 任务引擎     │→│ SSH 分发器   │→│ 判定引擎     │ │
│  └──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘ │
│        │                │                │                │          │
│        ▼                ▼                ▼                ▼          │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐ │
│  │ docs/        │  │ TaskQueue    │  │ SSH Pool     │  │ Reporter     │ │
│  │ contracts/   │  │ (Redis)      │  │ 4台机器连接  │  │ 钉钉+日志    │ │
│  └──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘ │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │                    State Machine（状态机）                 │    │
│  │  IDLE → PARSING → DISPATCHING → CODING → REVIEWING        │    │
│  │  → TESTING → JUDGING → [PASS|RETRY|ESCALATE]              │    │
│  └──────────────────────────────────────────────────────────┘    │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

2.2 核心模块说明

2.2.1 Doc Parser（文档解析器）

从 Git 仓库读取设计文档和 Sprint 任务卡，解析为结构化任务列表：

```python
# orchestrator/doc_parser.py — 伪代码

class DocParser:
    """解析设计文档 + Sprint 任务卡 → 生成编码任务"""

    def __init__(self, repo_path: str, llm_client):
        self.repo_path = repo_path
        self.llm = llm_client  # 连接 4090 上的 vLLM

    def parse_sprint_tasks(self, sprint_id: int) -> list[CodingTask]:
        """
        读取 07-Sprint任务卡.md 中指定 Sprint 的内容,
        结合 04-系统概要设计.md 中的架构描述,
        生成可执行的编码任务列表。
        """
        sprint_card = self._read_doc(f"docs/07-Sprint任务卡.md")
        design_doc = self._read_doc(f"docs/04-系统概要设计.md")
        contracts = self._read_contracts()

        prompt = f"""
        你是一个任务拆解引擎。请根据以下文档, 将 Sprint {sprint_id} 的任务
        拆解为独立的编码任务。

        每个任务需包含:
        - task_id: 唯一标识
        - target_machine: 目标机器 (4090/mac_min_8T/gateway/data_center)
        - target_dir: 目标目录
        - description: 任务描述 (给 aider 的详细指令)
        - context_files: 需要 aider 读取的上下文文件
        - depends_on: 依赖的前置任务 ID 列表
        - acceptance: 自动验收标准 (用于 Review 判定)

        === Sprint 任务卡 ===
        {sprint_card}

        === 系统设计 (相关章节) ===
        {design_doc}

        === 接口契约 ===
        {contracts}
        """
```

```
        tasks = self.llm.generate(prompt, response_format="json")
        return [CodingTask(**t) for t in tasks]
```

2.2.2 Task Engine（任务引擎）

管理任务队列、依赖关系和执行顺序：

```python
# orchestrator/task_engine.py — 伪代码

class TaskEngine:
    """任务调度引擎：维护队列、处理依赖、管理重试"""

    def __init__(self, redis_client):
        self.queue = redis_client       # Redis 作为任务队列
        self.max_retries = 3            # 每个任务最多重试 3 次
        self.concurrent_limit = 4       # 最多 4 台机器同时编码

    def enqueue(self, tasks: list[CodingTask]):
        """将任务入队，按依赖关系排序"""
        ordered = self._topological_sort(tasks)
        for task in ordered:
            self.queue.rpush("task:pending", task.to_json())

    def next_batch(self) -> list[CodingTask]:
        """
        取出下一批可并行执行的任务：
        - 无未完成的前置依赖
        - 目标机器不冲突（每台机器同时只执行 1 个任务）
        - 最多 4 个任务并行
        """
        pending = self._get_pending()
        completed = self._get_completed_ids()
        batch = []
        used_machines = set()

        for task in pending:
            if all(dep in completed for dep in task.depends_on):
                if task.target_machine not in used_machines:
                    batch.append(task)
                    used_machines.add(task.target_machine)
            if len(batch) >= self.concurrent_limit:
                break
        return batch

    def handle_result(self, task_id: str, result: TaskResult):
        """处理任务结果：通过/重试/升级"""
        if result.passed:
            self._mark_completed(task_id)
        elif result.retry_count < self.max_retries:
            self._requeue_with_fix(task_id, result.fix_instruction)
```

```
        else:
            self._escalate(task_id, result)  # 钉钉通知人工
```

2.2.3 Dispatcher（SSH 分发器）

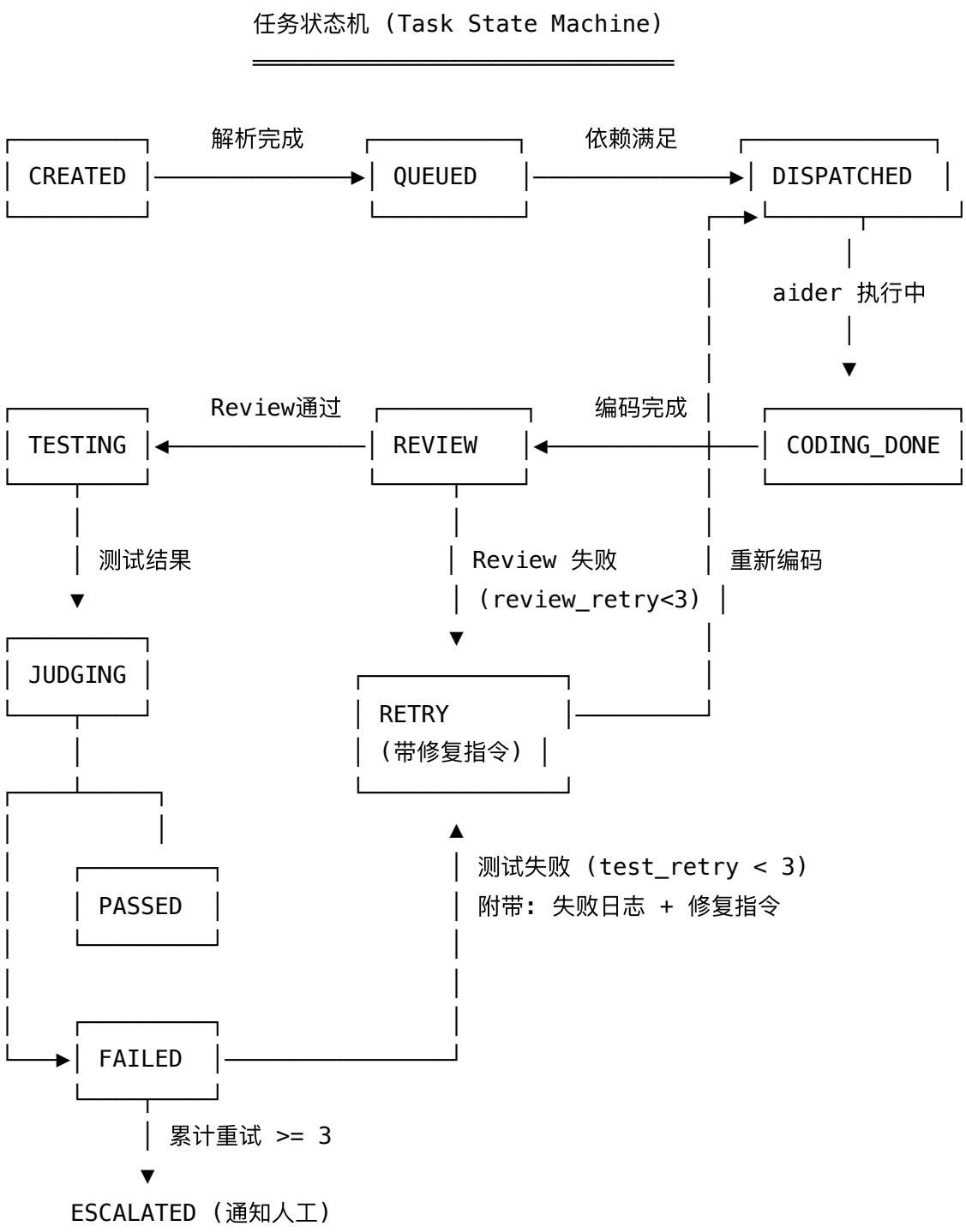
通过 SSH 在远程机器上执行 aider 编码任务：

```python
# orchestrator/dispatcher.py — 伪代码

import asyncio
import asyncssh

class Dispatcher:
    """SSH 分发器：将编码任务推送到目标机器执行"""

    SSH_CONFIG = {
        "4090":         {"host": "172.16.11.194", "user": "user"},
        "mac_min_8T":   {"host": "172.16.12.50",  "user": "edge_sale"},
        "gateway":      {"host": "172.16.14.215", "user": "realman"},
        "data_center":  {"host": "172.16.14.90",  "user": "realman"},
    }

    async def dispatch_batch(self, tasks: list[CodingTask]) -> list[TaskResult]:
        """并行分发一批任务到不同机器"""
        coros = [self._execute_on_remote(task) for task in tasks]
        return await asyncio.gather(*coros)

    async def _execute_on_remote(self, task: CodingTask) -> TaskResult:
        """SSH 到目标机器，启动 aider 执行编码任务"""
        cfg = self.SSH_CONFIG[task.target_machine]

        async with asyncssh.connect(cfg["host"], username=cfg["user"]) as conn:
            # 1. 确保代码最新
            await conn.run("cd ~/ai-crm && git pull --rebase")

            # 2. 写入任务指令文件
            instruction = self._build_aider_instruction(task)
            await conn.run(f"cat > /tmp/task_{task.task_id}.md << 'TASK_EOF'\n"
                           f"{instruction}\n"
                           f"TASK_EOF")

            # 3. 执行 aider
            result = await conn.run(
                f"cd ~/ai-crm && "
                f"aider --model openai/claude-opus-4-6 "
                f"--openai-api-base {os.environ['OPENAI_API_BASE']} "
                f"--openai-api-key {os.environ['OPENAI_API_KEY']} "
                f"--message-file /tmp/task_{task.task_id}.md "
                f"--yes-always "        # 自动确认所有操作
                f"--auto-commits "      # 自动 commit
```

```
        f"--no-suggest-shell-commands "
        f"{' '.join(task.context_files)}",
        timeout=600             # 单任务最长 10 分钟
    )

    # 4. 收集结果
    return TaskResult(
        task_id=task.task_id,
        stdout=result.stdout,
        exit_code=result.exit_code,
        files_changed=self._parse_changed_files(result.stdout),
    )
```

2.3 任务状态机

每个编码任务经历以下状态流转：



关键变更：测试失败（ `FAILED` ）不再直接升级为人工处理，而是将失败日志 + 错误堆栈 + 自动生成的修复指令一起注入 `RETRY` 状态，由 aider 重新编码修复后再走 Review → 测试 完整流程，直至累计重试次数（review_retry + test_retry）≥ 3 才升级。

2.4 Orchestrator 主循环

```python
# orchestrator/main.py — 主循环伪代码

class Orchestrator:
    """主调度循环：持续运行，自动执行开发流水线"""

    def __init__(self):
        self.parser = DocParser(repo_path="~/ai-crm", llm_client=cloud_api_client)  # 云
        self.engine = TaskEngine(redis_client=redis)
        self.dispatcher = Dispatcher()
        self.reviewer = AutoReviewer(llm_client=cloud_api_client)   # §4 详述，云端API
        self.reporter = Reporter(dingtalk_webhook=WEBHOOK_URL)  # §6 详述

    async def run_sprint(self, sprint_id: int):
        """执行一个完整 Sprint 的自动化开发"""
        # 1. 解析 Sprint 任务
        tasks = self.parser.parse_sprint_tasks(sprint_id)
        self.engine.enqueue(tasks)
        self.reporter.notify(f"Sprint {sprint_id} 启动，共 {len(tasks)} 个任务")

        # 2. 持续执行直到所有任务完成
        while not self.engine.all_done():
            # 取下一批可并行的任务
            batch = self.engine.next_batch()
            if not batch:
                await asyncio.sleep(30)  # 等待依赖完成
                continue

            # 3. 分发执行
            results = await self.dispatcher.dispatch_batch(batch)

            # 4. 自动 Review
            for result in results:
                review = await self.reviewer.review(result)
                if review.passed:
                    # 5. git push → 触发 orchestrator 本地自动测试
                    await self._push_and_test(result)
                else:
                    self.engine.handle_result(result.task_id, review)

            # 6. 每轮报告
            self.reporter.round_summary(batch, results)
```

```python
    # 7. Sprint 完成
    self.reporter.sprint_summary(sprint_id)

async def run_continuous(self):
    """持续运行模式：监听文档变更，自动触发任务"""
    while True:
        if self._docs_changed():
            # 文档更新 → 重新解析 → 增量生成新任务
            new_tasks = self.parser.parse_incremental()
            self.engine.enqueue(new_tasks)

        # 执行待处理的任务
        batch = self.engine.next_batch()
        if batch:
            results = await self.dispatcher.dispatch_batch(batch)
            for result in results:
                review = await self.reviewer.review(result)
                self.engine.handle_result(result.task_id, review)

        await asyncio.sleep(10)  # 10 秒轮询
```

2.5 Orchestrator 配置文件

```yaml
# orchestrator/config.yaml

orchestrator:
  mode: sprint          # sprint | continuous
  current_sprint: 1
  poll_interval: 10     # 秒

llm:
  base_url: http://172.16.11.194:8000/v1
  model: Qwen3-30B-A3B
  api_key: dummy
  timeout: 120
  max_tokens: 8192

machines:
  4090:
    host: 172.16.11.194
    user: user
    work_dir: ~/ai-crm
    owned_dirs: [agent/]
    aider_model: openai/claude-opus-4-6

  mac_min_8T:
    host: 172.16.12.50
    user: edge_sale
    work_dir: ~/ai-crm
    owned_dirs: [crm/, frontend/]
    aider_model: openai/claude-opus-4-6

  gateway:
    host: 172.16.14.215
    user: realman
    work_dir: ~/ai-crm
    owned_dirs: [deploy/nginx/, deploy/dingtalk/]
    aider_model: openai/claude-opus-4-6

  data_center:
    host: 172.16.14.90
    user: realman
    work_dir: ~/ai-crm
    owned_dirs: [deploy/monitoring/, scripts/]
```

```yaml
        aider_model: openai/claude-opus-4-6

llm_gateway:
    openai_api_base: ${OPENAI_API_BASE}     # 见 .env
    openai_api_key: ${OPENAI_API_KEY}       # 见 .env

task:
  max_retries: 3
  single_task_timeout: 600    # 秒
  concurrent_limit: 4
  batch_push_interval: 3      # 每 3 个任务推送一次

git:
  bare_repo: ssh://edge_sale@172.16.12.50/opt/git/ai-crm.git
  branch: main
  auto_tag_on_sprint_end: true

testing:
  host: localhost              # 测试在 orchestrator 本地执行，零网络延迟
  trigger: post-receive-hook   # mac_min_8T 裸仓库触发
  report_path: ~/ai-crm/reports/
  pass_threshold: 4.0          # 云端多模型API加权评分通过线（GPT-4.1×0.40 + Qwen-Max×0.35
  vllm_endpoint: http://172.16.11.194:8000/v1  # 4090 vLLM A（Agent 测试推理，内网调用）

notification:
  dingtalk_webhook: ${DINGTALK_WEBHOOK_URL}
  notify_on: [sprint_start, sprint_end, task_escalated, daily_summary]
  daily_summary_time: "18:00"
```

2.6 目录结构

```
ai-crm/
├── orchestrator/                    ← Orchestrator 源码（专用服务器运行）
│   ├── __init__.py
│   ├── main.py                      ← 主循环入口
│   ├── config.yaml                  ← 配置文件
│   ├── doc_parser.py                ← 文档解析器
│   ├── task_engine.py               ← 任务引擎
│   ├── task_models.py               ← CodingTask / TaskResult 数据模型
│   ├── dispatcher.py                ← SSH 分发器
│   ├── reviewer.py                  ← 自动 Review 引擎（§4）
│   ├── reporter.py                  ← 报告与通知（§6）
│   ├── git_ops.py                   ← Git 自动化操作（§5）
│   └── state_machine.py             ← 任务状态机
├── contracts/                       ← 接口契约（你维护）
├── docs/                            ← 设计文档（你维护）
├── agent/                           ← 4090 编码
├── crm/                             ← mac_min_8T 编码
├── frontend/                        ← mac_min_8T 编码
├── deploy/                          ← gateway + data_center 编码
├── scripts/                         ← data_center 编码
├── tests/                           ← orchestrator 本地执行
└── reports/                         ← 自动生成的测试/进度报告
```

3. 自动化编码引擎

每台开发机器上运行 **aider**（AI 编码 CLI），通过 OpenAI 兼容 API 连接 Claude Opus 4.6，实现自动代码生成与提交。AutoReview 和 DocParser 走云端 API（强模型更适合代码审查和文档解析），Agent 集成测试推理统一调用 4090 上的 **vLLM 实例 A**（内网 <1ms）。

3.1 aider 简介与选型理由

维度	aider	自定义脚本	Copilot CLI
OpenAI 兼容 API	✅ 原生支持	需自行实现	❌ 不支持本地模型
上下文管理	✅ 自动分析 repo map	需手动指定	有限

维度	aider	自定义脚本	Copilot CLI
自动 Git commit	✅ 内置	需自行实现	❌
多文件编辑	✅ diff 格式精准	容易出错	有限
无交互模式	✅ `--yes-always`	天然	❌ 需交互
社区活跃度	✅ 70k+ stars	-	-

> aider 是目前最成熟的 AI 编码 CLI，原生支持 OpenAI 兼容 API，可直接连接 vLLM。

3.2 aider 安装与配置

4 台开发机器均需安装 aider（全部通过一步 API 网关连接 Claude Opus 4.6）：

```bash
#!/bin/bash
# scripts/setup_aider.sh — 在目标机器上安装 aider

set -e

# 1. 安装 aider
pip install aider-chat

# 2. 配置 aider（连接一步 API 的 OpenAI 兼容网关）
mkdir -p ~/.config/aider
cat > ~/.aider.conf.yml << 'EOF'
# aider 全局配置
model: openai/claude-opus-4-6
openai-api-base: ${OPENAI_API_BASE}     # 见 .env
openai-api-key: ${OPENAI_API_KEY}       # 见 .env

# 自动化模式
yes-always: true           # 自动确认所有操作
auto-commits: true         # 每次编辑自动 commit
no-suggest-shell-commands: true
no-auto-lint: false        # 启用自动 lint
stream: false              # 非流式（脚本调用更稳定）

# 代码风格
code-theme: monokai
edit-format: diff           # 使用 diff 格式（比 whole 更精准）

# 上下文管理
map-tokens: 2048            # repo map 使用的 token 数
map-refresh: auto           # 自动刷新 repo map
EOF

# 3. 验证连接
echo "验证 aider 连接 vLLM..."
cd ~/ai-crm
echo "/exit" | aider --message "回复OK确认连接正常" --no-auto-commits 2>&1 | head -20
echo "✅ aider 安装配置完成"
```

一键部署到 4 台机器：

```
# 在 data_center 上执行：将 aider 安装到所有开发机
for host in 4090 mac_min_8T gateway data_center; do
    echo "=== 安装 aider 到 $host ==="
    ssh $host "bash -s" < scripts/setup_aider.sh
done
```

3.3 编码任务定义格式

Orchestrator 生成的编码任务遵循统一格式，写入临时指令文件交给 aider：

```yaml
# 编码任务定义 (CodingTask)

task_id: "S1-D3-W1-001"          # Sprint1-Day3-Window1-序号
sprint: 1
day: 3
target_machine: "4090"
target_dir: "agent/"
priority: 1                       # 1=最高, 5=最低

description: |
  根据 docs/04-系统概要设计.md §5.1 实现 Supervisor Agent 的意图识别逻辑。

  要求:
  1. 参考 contracts/agent-api.yaml 中的 /agent/chat 接口定义
  2. 使用 FastAPI + vLLM Tool Calling
  3. 支持的意图类型: create_lead, query_lead, update_lead, query_customer
  4. 意图识别失败时返回友好提示"我没理解您的意思, 请再说详细一点"
  5. 包含完整的错误处理和日志记录
  6. 添加 docstring 说明
  7. 在 tests/ 目录下生成对应的 pytest 测试文件

context_files:                    # aider 需要加载的上下文文件
  - contracts/agent-api.yaml
  - contracts/agent-tools.yaml
  - agent/__init__.py             # 如果已存在

depends_on:                       # 前置依赖
  - "S1-D2-W1-001"               # Agent 框架骨架

acceptance:                       # 自动验收标准
  - "agent/supervisor.py 文件存在"
  - "包含 class SupervisorAgent"
  - "包含意图识别方法 identify_intent()"
  - "包含 4 种意图类型的处理分支"
  - "tests/test_supervisor.py 存在"
  - "代码可通过 python -m py_compile 编译检查"

output_files:                     # 预期产出文件
  - agent/supervisor.py
  - agent/prompts/supervisor_system.jinja2
  - tests/test_supervisor.py
```

3.4 aider 执行指令构建

Orchestrator 的 Dispatcher 将 CodingTask 转换为 aider 命令：

```python
# orchestrator/dispatcher.py — 构建 aider 执行指令

def _build_aider_instruction(self, task: CodingTask) -> str:
    """将结构化任务转换为 aider 可理解的自然语言指令"""

    instruction = f"""
# 编码任务 {task.task_id}

## 目标
{task.description}

## 关键约束
- 严格遵循 contracts/ 中的接口定义，函数签名、参数名、返回值类型必须一致
- 所有配置项（IP、端口、密码）从环境变量或 config 文件读取，禁止硬编码
- 包含完整的异常处理（try/except），错误信息要有上下文
- 每个公开方法都要有 docstring
- import 语句完整，确保代码可独立运行
- 同时生成对应的 pytest 测试文件

## 验收标准
{chr(10).join(f'- {a}' for a in task.acceptance)}

## 代码风格
- Python: PEP 8, type hints, async/await
- 缩进: 4 空格
- 字符串: 优先 f-string
- 日志: 使用 loguru
"""
    return instruction

def _build_aider_command(self, task: CodingTask) -> str:
    """构建完整的 aider CLI 命令"""
    context_args = " ".join(task.context_files)
    return (
        f"cd ~/ai-crm && "
        f"aider "
        f"--message-file /tmp/task_{task.task_id}.md "
        f"--yes-always "
        f"--auto-commits "
        f"--no-suggest-shell-commands "
        f"--commit-prompt 'feat({task.target_dir.rstrip('/')}): {task.task_id}' "
```

```
        f"{context_args}"
    )
```

3.5 各机器的编码职责与典型任务

机器	目录	Sprint 1-2 典型任务	Sprint 3-4 典型任务
4090	`agent/`	Agent 框架骨架、Supervisor Agent、vLLM Tool Calling 封装	销售助手 Agent、Prompt 模板、Agent 间通信
mac_min_8T	`crm/` `frontend/`	FastAPI 骨架、DB Schema、Alembic 迁移、SQLAlchemy 模型	线索/客户 CRUD API、Pydantic 模型、Vue3 列表页
gateway	`deploy/nginx/` `deploy/dingtalk/`	Nginx 反向代理配置、SSL 证书、健康检查端点	钉钉机器人回调、SSO 集成、互动卡片模板
data_center	`deploy/monitoring/` `scripts/`	Prometheus + Grafana 配置、PG 备份脚本、日志收集	Celery 任务、日报生成、数据同步脚本

3.6 编码过程的自动 commit 规范

aider 自动生成的 commit 遵循统一格式：

```
<type>(<scope>): <task_id> <简要描述>

type:
  feat      新功能
  fix       Bug 修复（含 Review 后的修复）
  refactor  重构
  test      测试文件

scope（对应机器）:
  agent     4090 – Agent 引擎
  crm       mac_min_8T – CRM 后端
  frontend  mac_min_8T – 前端
  nginx     gateway – 网关
  dingtalk  gateway – 钉钉
  ops       data_center – 运维/监控

示例:
  feat(agent): S1-D3-W1-001 实现 Supervisor Agent 意图识别
  fix(crm): S1-D3-W2-001 修复 leads API 分页参数（retry 2/3）
  test(agent): S1-D3-W1-001 新增 Supervisor Agent 单元测试
```

3.7 编码超时与异常处理

```
                    编码阶段异常处理
                    ═══════════════

┌── 正常路径 ──────────────────────────────────────┐
│  aider 启动 → 生成代码 → 自动 commit → 返回结果   │
│  耗时: 1-5 分钟                                   │
└──────────────────────────────────────────────────┘

┌── 超时 (>10min) ────────────────────────────────┐
│  kill aider 进程                                 │
│  → 检查是否有部分 commit                         │
│  → 有: 标记为 CODING_DONE, 进入 Review           │
│  → 无: 拆分为更小的子任务重新排队                │
└──────────────────────────────────────────────────┘

┌── SSH 连接失败 ──────────────────────────────────┐
│  重试 3 次 (间隔 30s)                            │
│  → 仍失败: 标记机器为 DOWN                       │
│  → 该机器的任务暂挂                              │
│  → 钉钉通知: "4090 SSH 不可达, XX个任务暂挂"     │
└──────────────────────────────────────────────────┘

┌── vLLM 不可用 ────────────────────────────────────┐
│  aider 报连接错误                                 │
│  → 检查 vLLM 状态: curl 172.16.11.194:8000/health │
│  → 不可用: 尝试 SSH 重启 vLLM (tmux send-keys)    │
│  → 等待 3 分钟模型加载                            │
│  → 仍不可用: 全部编码任务暂停 + 告警              │
└──────────────────────────────────────────────────┘

┌── aider 生成垃圾代码 ────────────────────────────┐
│  进入 Review 阶段自动检测                        │
│  → Review 不通过 → 带修复指令重试 (最多 3 次)    │
│  → 3 次仍垃圾 → 升级人工                         │
└──────────────────────────────────────────────────┘
```

4. 自动 Review 与迭代循环

编码完成后，代码不直接推送到主仓库，而是先经过 **LLM 自动 Review**。Review 对照设计文档和接口契约检查代码质量，不通过则自动生成修复指令并重试。

4.1 自动 Review 的三层检查

```
                    自动 Review 三层检查
                    ═══════════════════

Layer 1: 静态检查 (0 成本, 秒级)
├── python -m py_compile *.py        ← 语法正确性
├── ruff check --select E,W,F        ← 代码规范
├── mypy --ignore-missing-imports    ← 类型检查 (宽松模式)
└── 结果: 有错 → 直接生成修复指令回到编码阶段

Layer 2: 契约对齐检查 (LLM 驱动)
├── 对比代码与 contracts/*.yaml 的接口签名
├── 检查函数名、参数名、返回类型是否一致
├── 检查缺失的接口方法
└── 结果: 不一致 → 生成具体的对齐修复指令

Layer 3: 设计符合度检查 (LLM 驱动)
├── 对比代码与 docs/04-系统概要设计.md 的设计意图
├── 检查核心逻辑是否正确实现
├── 检查是否有硬编码、缺少异常处理、缺少日志
├── 检查是否有遗漏的功能点
└── 结果: 评分 + 具体修改建议
```

4.2 AutoReviewer 实现

```python
# orchestrator/reviewer.py — 自动 Review 引擎

class AutoReviewer:
    """三层自动代码 Review"""

    def __init__(self, llm_client, repo_path: str):
        self.llm = llm_client
        self.repo_path = repo_path

    async def review(self, result: TaskResult, task: CodingTask) -> ReviewResult:
        """对编码结果执行三层 Review"""

        # Layer 1: 静态检查
        static_issues = await self._static_check(result.files_changed)
        if static_issues:
            return ReviewResult(
                passed=False,
                layer="static",
                issues=static_issues,
                fix_instruction=self._static_fix_instruction(static_issues)
            )

        # Layer 2: 契约对齐
        contract_issues = await self._contract_check(result, task)
        if contract_issues:
            return ReviewResult(
                passed=False,
                layer="contract",
                issues=contract_issues,
                fix_instruction=contract_issues  # LLM 已生成修复指令
            )

        # Layer 3: 设计符合度
        design_review = await self._design_check(result, task)
        if design_review.score < 4.0:
            return ReviewResult(
                passed=False,
                layer="design",
                issues=design_review.issues,
                fix_instruction=design_review.fix_instruction,
                score=design_review.score
```

```
            )

        return ReviewResult(passed=True, score=design_review.score)

    async def _static_check(self, files: list[str]) -> list[str]:
        """Layer 1: 静态检查"""
        issues = []
        for f in files:
            if f.endswith(".py"):
                # 编译检查
                result = await self._run_remote(
                    f"cd ~/ai-crm && python -m py_compile {f}"
                )
                if result.exit_code != 0:
                    issues.append(f"编译错误 {f}: {result.stderr}")

                # ruff 检查
                result = await self._run_remote(
                    f"cd ~/ai-crm && ruff check {f} --select E,W,F"
                )
                if result.exit_code != 0:
                    issues.append(f"Lint 问题 {f}: {result.stdout}")
        return issues

    async def _contract_check(self, result: TaskResult, task: CodingTask) -> str:
        """Layer 2: 对照接口契约检查"""
        code_content = self._read_files(result.files_changed)
        contracts = self._read_contracts(task)

        prompt = f"""
你是一个接口契约审查员。请对比以下代码和接口契约，检查是否一致。

## 接口契约
{contracts}

## 生成的代码
{code_content}

## 检查项
1. 函数/方法签名是否和契约定义一致（名称、参数、返回类型）？
2. 是否有契约中定义但代码中未实现的接口?
3. 参数的校验逻辑是否和契约中的约束一致（required、enum、type）？
```

```
## 输出格式
如果完全一致，返回: {{"passed": true}}
如果有不一致，返回:
{{
    "passed": false,
    "issues": ["问题描述1", "问题描述2"],
    "fix_instruction": "具体的修复指令，告诉 aider 如何修改"
}}
"""
        result = self.llm.generate(prompt, response_format="json")
        if not result["passed"]:
            return result["fix_instruction"]
        return None

    async def _design_check(self, result: TaskResult, task: CodingTask) -> DesignReview
        """Layer 3: 对照设计文档检查"""
        code_content = self._read_files(result.files_changed)
        design_doc = self._read_relevant_design_sections(task)

        prompt = f"""
你是一个高级代码审查员。请根据设计文档评审以下代码。

## 设计文档（相关章节）
{design_doc}

## 编码任务
{task.description}

## 验收标准
{chr(10).join(task.acceptance)}

## 生成的代码
{code_content}

## 评审维度（每项 1-5 分）
1. **功能完整性**: 是否完整实现了设计文档要求的所有功能点?
2. **接口正确性**: 函数签名、参数、返回值是否符合契约?
3. **错误处理**: 是否有完整的异常处理? 错误信息是否有上下文?
4. **代码质量**: 是否有硬编码? docstring 是否完整? 日志是否充分?
5. **可运行性**: import 是否齐全? 是否可以独立运行/测试?

## 输出格式（JSON）
{{
```

```
    "scores": {{"功能完整性": 4, "接口正确性": 5, ...}},
    "average_score": 4.2,
    "issues": ["问题1", "问题2"],
    "fix_instruction": "如果平均分<4.0，给出具体修复指令"
}}
"""
        return self.llm.generate(prompt, response_format="json")
```

4.3 迭代修复循环

当 Review 不通过时，系统自动生成修复指令并重试编码：

```
                    迭代修复循环 (Fix Loop)
                    ═══════════════════════

Round 1: 初始编码

┌──────────────────────────────────────────────────┐
│  aider 根据原始任务描述生成代码                    │
│  → 自动 commit: "feat(agent): S1-D3-W1-001 ..."  │
│  → 进入 Review                                   │
│  → Review 结果: ❌ 缺少 query_customer 意图处理   │
└──────────────────────────────────────────────────┘
                        │
                        ▼ 修复指令自动生成
Round 2: 第 1 次修复

┌──────────────────────────────────────────────────┐
│  aider 收到修复指令:                              │
│  "在 supervisor.py 的 identify_intent() 中        │
│   添加 query_customer 意图的处理分支,             │
│   参考 contracts/agent-tools.yaml 中的定义"       │
│  → 自动 commit: "fix(agent): S1-D3-W1-001        │
│     添加 query_customer 意图 (retry 1/3)"         │
│  → 进入 Review                                   │
│  → Review 结果: ❌ 函数返回类型和契约不一致        │
└──────────────────────────────────────────────────┘
                        │
                        ▼ 修复指令自动生成
Round 3: 第 2 次修复

┌──────────────────────────────────────────────────┐
│  aider 收到修复指令:                              │
│  "修改 identify_intent() 的返回类型, 从 str       │
│   改为 IntentResult(intent: str, confidence:      │
│   float, slots: dict), 和 agent-api.yaml 一致"    │
│  → 自动 commit: "fix(agent): S1-D3-W1-001        │
│     修复返回类型 IntentResult (retry 2/3)"        │
│  → 进入 Review                                   │
│  → Review 结果: ✅ 通过 (score: 4.3)              │
└──────────────────────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────────┐
│  git push → orchestrator 本地自动测试             │
│  → pytest + 云端API多模型 Judge (GPT-4.1/Qwen-Max/DeepSeek-V3) │
```

```
│   → 测试通过 → 任务 PASSED → 取下一个任务                    │
└──────────────────────────────────────────────────────────────┘
```

4.4 修复指令的生成策略

```
# orchestrator/reviewer.py — 修复指令生成

def generate_fix_instruction(self, task: CodingTask,
                             review: ReviewResult,
                             retry_count: int) -> str:
    """根据 Review 结果和重试次数生成递进式修复指令"""

    if retry_count == 0:
        # 第 1 次修复：精准修复 Review 发现的问题
        return f"""
请修复以下 Review 发现的问题：

{chr(10).join(f'- {issue}' for issue in review.issues)}

只修改有问题的部分，不要重写整个文件。
确保修复后代码仍然可以通过编译检查。
"""

    elif retry_count == 1:
        # 第 2 次修复：带更多上下文和示例
        return f"""
上一次修复不完整。请重新检查以下问题：

{chr(10).join(f'- {issue}' for issue in review.issues)}

参考以下契约定义，确保接口严格一致：
{self._read_relevant_contract(task)}

特别注意：
- 函数签名必须和契约完全一致
- 返回值结构必须和契约定义的 schema 匹配
- 所有 required 字段都要有校验
"""

    else:
        # 第 3 次修复：最后机会，提供完整重写指引
        return f"""
这是最后一次自动修复机会。之前的修复均未完全解决问题。

遗留问题：
{chr(10).join(f'- {issue}' for issue in review.issues)}
```

请考虑重新实现相关函数。以下是完整的参考信息：

设计要求：
{task.description}

接口契约：
{self._read_relevant_contract(task)}

验收标准：
{chr(10).join(f'- {a}' for a in task.acceptance)}

如果无法一次性解决所有问题，优先确保：
1. 代码可编译运行
2. 接口签名和契约一致
3. 核心逻辑正确
"""

4.5 Review + 测试的完整闭环

```
                    Review + 测试完整闭环
                    ═══════════════════

┌─ Orchestrator (专用服务器) ──────────────────────────────────┐
│                                                              │
│  编码完成 ──▶ Layer 1 静态检查                               │
│                  │                                           │
│            ┌── PASS ──┐                                      │
│            │          ▼                                      │
│            │   Layer 2 契约检查                              │
│            │          │                                      │
│            │     ┌── PASS ──┐                               │
│            │     │          ▼                               │
│            │     │   Layer 3 设计检查                       │
│            │     │          │                               │
│            │     │     ┌── score ≥ 4.0 ──┐                 │
│            │     │     │                  ▼                 │
│            │     │     │          git push to repo          │
│            │     │     │                  │                 │
│            │     │     │        post-receive hook           │
│            │     │     │                  │                 │
│            │     │     │                  ▼                 │
│            │     │     │   ┌─ orchestrator 本地测试 ──────┐ │
│            │     │     │   │                              │ │
│            │     │     │   │  pytest (单元+集成)          │ │
│            │     │     │   │  → 云端API多模型Judge(业务评估)│ │
│            │     │     │   │  → 生成测试报告              │ │
│            │     │     │   │                              │ │
│            │     │     │   └──────────────────────────────┘ │
│            │     │     │              │                     │
│            │     │     │          测试结果                  │
│            │     │     │              │                     │
│            │     │     │    ┌── 全绿 ──┼── 有红灯 ──┐      │
│            │     │     │    │          │            │      │
│            │     │     │    ▼          │            ▼      │
│            │     │     │  PASSED       │     生成修复指令   │
│            │     │     │  下一个任务   │     → 重新编码     │
│            │     │     │               │                   │
│       FAIL │ FAIL│     │ FAIL          │                   │
│            ▼     ▼     ▼               │                   │
│       生成修复指令 → aider 重新编码 ──┘                     │
```

```
│        （最多 3 次）                                                  │
│                                                                       │
└───────────────────────────────────────────────────────────────────────┘
```

5. Git 自动化与持续部署

5.1 Git 工作流全景

整个流水线的 Git 操作完全自动化，无需人工执行任何 git 命令（除了你推送设计文档更新）：

```
                          Git 自动化全景
                          ═══════════════

┌─ 你（唯一的人工 Git 操作）──────────────────────────────────────┐
│                                                                  │
│  vim docs/04-系统概要设计.md      ← 修改设计文档                 │
│  vim contracts/agent-api.yaml    ← 更新接口契约                  │
│  git add . && git commit && git push                             │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
                            │
                            │
                  ┌─────────▼──────────┐
                  │ Git 裸仓库          │
                  │ mac_min_8T:         │
                  │ /opt/git/ai-crm.git │
                  └────────────────────┘
                            │
                            │
            ┌───────────────┼──────────────────┐
            │               │                  │
  post-receive       Orchestrator       post-receive
  → orchestrator    检测 docs/ 变更     → 4台开发机
  （本地测试触发）   → 重新生成任务      （代码同步）
```

5.2 自动 commit + push 流程

```python
# orchestrator/git_ops.py — Git 自动化操作

class GitOps:
    """Git 自动化：commit, push, pull, tag"""

    def __init__(self, dispatcher: Dispatcher):
        self.dispatcher = dispatcher

    async def push_from_machine(self, machine: str, task: CodingTask):
        """
        从指定机器 push 代码到中央仓库。

        策略：串行提交 + 人工确认（R-013 决议）
        - aider 编码后自动 commit，但 push 需人工确认
        - 共享文件（requirements.txt、config.py）仅在 mac_min_8T 上修改
        - 按子目录隔离（agent/ / crm/ / deploy/），冲突概率极低
        """
        cfg = Dispatcher.SSH_CONFIG[machine]
        async with asyncssh.connect(cfg["host"], username=cfg["user"]) as conn:
            # aider 已经自动 commit，push 前通知人工确认
            await self.dispatcher.notify(
                f"🔔 {machine} 编码完成，请确认后 push",
                task_summary=task.description
            )
            result = await conn.run(
                "cd ~/ai-crm && git push origin main",
                timeout=60
            )
            if result.exit_code != 0:
                # push 失败 → 通知人工处理（不自动 rebase）
                await self.dispatcher.notify(
                    f"⚠️ {machine} push 失败，请手动处理",
                    error=result.stderr
                )

    async def sync_all_machines(self):
        """所有机器 pull 最新代码"""
        coros = []
        for machine in ["4090", "mac_min_8T", "gateway", "data_center"]:
            coros.append(self._pull_on_machine(machine))
        await asyncio.gather(*coros)
```

```python
    async def auto_tag_sprint(self, sprint_id: int):
        """Sprint 结束时自动打 tag"""
        version = f"v0.{sprint_id // 2 + 1}.0"
        async with asyncssh.connect(
            "172.16.12.50", username="edge_sale"
        ) as conn:
            await conn.run(
                f"cd ~/ai-crm && "
                f"git tag -a {version} -m 'Sprint {sprint_id} 完成' && "
                f"git push origin {version}"
            )

    # 注：原有 _handle_push_failure 方法（自动 rebase + GitConflictError）已移除
    # R-013 决议：1 人团队采用串行提交 + 人工确认，不需要自动冲突处理
    # 如 push 失败，通知开发者手动处理即可
```

5.3 post-receive Hook（测试自动触发）

Git 裸仓库的 post-receive hook 自动触发 orchestrator 本地测试：

```bash
#!/bin/bash
# /opt/git/ai-crm.git/hooks/post-receive
# 代码推送后自动触发测试

set -e

# 读取推送信息
while read oldrev newrev refname; do
    branch=$(git rev-parse --symbolic --abbrev-ref $refname)

    if [ "$branch" = "main" ]; then
        echo "=== main 分支更新，触发自动化流水线 ==="

        # 1. 获取变更文件列表
        CHANGED_FILES=$(git diff --name-only $oldrev $newrev)
        echo "变更文件: $CHANGED_FILES"

        # 2. 判断变更类型
        HAS_CODE_CHANGE=false
        HAS_DOC_CHANGE=false

        echo "$CHANGED_FILES" | grep -qE "^(agent|crm|frontend|deploy|scripts)/" && HAS
        echo "$CHANGED_FILES" | grep -qE "^(docs|contracts)/" && HAS_DOC_CHANGE=true

        # 3. 代码变更 → 触发 orchestrator 本地测试
        if [ "$HAS_CODE_CHANGE" = true ]; then
            echo "代码变更，触发 orchestrator 本地自动测试..."
            ssh orchestrator "cd ~/ai-crm && \
                git pull && \
                nohup bash scripts/run_tests.sh > logs/test_$(date +%Y%m%d_%H%M%S).log
        fi

        # 4. 文档变更 → 通知 Orchestrator 重新解析
        if [ "$HAS_DOC_CHANGE" = true ]; then
            echo "文档变更，通知 Orchestrator..."
            ssh data_center "curl -s http://localhost:9500/api/docs-changed \
                -H 'Content-Type: application/json' \
                -d '{\"files\": \"$CHANGED_FILES\"}'"
        fi

        # 5. 同步到所有开发机
        for host in 4090 mac_min_8T gateway data_center; do
            ssh $host "cd ~/ai-crm && git pull --rebase" &
```

```
        done
        wait
        echo "=== 所有机器已同步 ===="
    fi
done
```

5.4 分支策略（不变）

一期仍采用**单分支策略**（main），与 v1.0 一致：

```
main ──●──●──●──●──●──●──●──●──●──●──●──●──●──●──●──▶
       |        |         |         |         |
       S1 开始   S1-2 标签  S3-4 标签  S5-6 标签  S7-8 标签
                v0.1.0    v0.2.0    v0.3.0    v1.0.0
```

```
自动化模式下的优势：
- 每台机器只改自己的目录（agent/ / crm/ / deploy/nginx/ / deploy/monitor/），冲突概率极低
- aider 自动 commit 确保每个变更都有记录
- 串行提交 + 人工确认 push，1 人团队无需分布式锁或自动 merge（R-013 决议）
- 共享文件（requirements.txt、config.py）仅在 mac_min_8T 上修改，避免多机竞争
- Sprint 结束自动打 tag
```

5.5 自动化部署：开发→运行模式切换

Sprint 7-8 完成后，Orchestrator 自动执行模式切换：

```bash
#!/bin/bash
# scripts/auto_deploy.sh — 自动部署脚本（Orchestrator 在 Sprint 8 末触发）

set -e
echo "=== 开始自动部署：开发模式 → 运行模式 ===="

# Step 1: 确认所有测试通过
echo "Step 1: 检查测试报告..."
REPORT=$(cat ~/ai-crm/reports/latest.json)  # orchestrator 本地读取，零网络延迟
SCORE=$(echo $REPORT | python3 -c "import sys,json; print(json.load(sys.stdin)['average
if (( $(echo "$SCORE < 4.0" | bc -l) )); then
    echo "❌ 测试评分 $SCORE < 4.0，中止部署"
    exit 1
fi
echo "✅ 测试评分 $SCORE ≥ 4.0"

# Step 2: 打 release tag
echo "Step 2: 打 release tag..."
ssh mac_min_8T "cd ~/ai-crm && \
    git tag -a v1.0.0 -m '一期正式发布' && \
    git push origin v1.0.0"

# Step 3: 切换到 Docker 部署
echo "Step 3: 构建 Docker 镜像..."
ssh mac_min_8T "cd ~/ai-crm && docker compose build"

echo "Step 4: 停止开发模式服务..."
ssh 4090 "tmux send-keys -t agent C-c"         # 停 Agent 开发模式
ssh mac_min_8T "tmux send-keys -t crm C-c"     # 停 CRM 开发模式

echo "Step 5: 启动运行模式..."
ssh mac_min_8T "cd ~/ai-crm && docker compose up -d"

echo "Step 6: 验证..."
sleep 10
for endpoint in \
    "172.16.11.194:8100/health" \
    "172.16.12.50:8900/health" \
    "172.16.14.215:80/health"; do
    if curl -sf "http://$endpoint" > /dev/null; then
        echo "✅ $endpoint"
    else
        echo "❌ $endpoint — 需要排查"
```

```
    fi
done

echo "Step 7: 通知钉钉..."
# 发送部署成功通知到钉钉群
curl -s "$DINGTALK_WEBHOOK_URL" \
    -H 'Content-Type: application/json' \
    -d '{
        "msgtype": "markdown",
        "markdown": {
            "title": "🚀 一期部署完成",
            "text": "## 🚀 睿尔曼智能 CRM 一期部署完成\n\n- 版本: v1.0.0\n- 测试评分: '"$SCOR
        }
    }'

echo "=== ✅ 部署完成 ==="
```

5.6 每批次 Git 操作时序

```
                    单批次 Git 时序
                    ═══════════════


T+0     Orchestrator 分发任务到 4 台机器
        │
T+5min  aider 编码完成，自动 commit（本地）
        │
T+6min  Orchestrator 自动 Review（三层检查）
        │
T+7min  ┌── Review 通过 ──────────────────────────────────┐
        │                                                  │
        │  Orchestrator 依次 push（避免冲突）:               │
        │  T+7:00  4090:        git push                   │
        │  T+7:05  mac_min_8T:  git pull && git push       │
        │  T+7:10  gateway:     git pull && git push       │
        │  T+7:15  data_center: git pull && git push       │
        │                                                  │
        │  T+7:20  post-receive hook 触发:                  │
        │          → orchestrator: 本地 pytest + 云端API Judge  │
        │          → 4 台开发机: git pull（获取别人的代码）   │
        │                                                  │
        └──────────────────────────────────────────────────┘
        │
T+9min  orchestrator 测试完成（本地执行，无网络延迟）
        │
T+10min Orchestrator 判定 → 取下一批任务
```

6. 监控报告与异常处理

6.1 三级报告体系

```
                    报告体系
                    ═══════

Level 1: 实时报告（每批次 ~15min）
├── 本批 4 个任务的执行结果
├── 通过/失败/重试统计
└── 输出到: Orchestrator 日志 + Redis

Level 2: 每日摘要（每天 18:00 自动发送）
├── 今日完成/失败/跳过的任务数
├── 代码行数变化（新增/删除/修改）
├── 测试通过率趋势
├── 云端多模型Judge 加权平均分（GPT-4.1/Qwen-Max/DeepSeek-V3）
└── 输出到: 钉钉群 + reports/daily/

Level 3: Sprint 报告（Sprint 结束时）
├── 里程碑达成情况
├── 全量测试通过率
├── 代码统计（模块分布、复杂度）
├── 风险项和遗留问题
└── 输出到: 钉钉群 + reports/sprint/ + docs/
```

6.2 Reporter 实现

```python
# orchestrator/reporter.py — 报告与通知引擎

import httpx
from datetime import datetime

class Reporter:
    """多级报告引擎：实时日志 + 钉钉通知 + 文件报告"""

    def __init__(self, dingtalk_webhook: str, report_dir: str):
        self.webhook = dingtalk_webhook
        self.report_dir = report_dir
        self.daily_stats = DailyStats()

    # ─── Level 1: 实时报告 ───

    def round_summary(self, batch: list[CodingTask], results: list[TaskResult]):
        """每批次执行后的实时摘要"""
        passed = sum(1 for r in results if r.passed)
        failed = len(results) - passed
        self.daily_stats.add_batch(passed, failed)

        log.info(f"批次完成: {passed}✅ {failed}❌ | "
                 f"任务: {[t.task_id for t in batch]}")

    # ─── Level 2: 每日摘要 ───

    async def daily_summary(self):
        """每天 18:00 自动发送到钉钉"""
        stats = self.daily_stats
        test_report = await self._fetch_test_report()

        markdown = f"""## 📊 每日开发摘要 — {datetime.now().strftime('%Y-%m-%d')}

### 任务执行
| 指标 | 数值 |
|------|------|
| 完成任务 | {stats.passed} |
| 失败任务 | {stats.failed} |
| 重试次数 | {stats.retries} |
| 升级人工 | {stats.escalated} |
```

```
### 代码变更
| 机器 | 新增行 | 修改行 | 删除行 |
|------|--------|--------|--------|
| 4090 (agent) | {stats.lines['4090']['added']} | {stats.lines['4090']['modified']} | {
| mac_min (crm) | {stats.lines['mac']['added']} | {stats.lines['mac']['modified']} | {s
| gateway | {stats.lines['gw']['added']} | {stats.lines['gw']['modified']} | {stats.lin
| data_center | {stats.lines['dc']['added']} | {stats.lines['dc']['modified']} | {stats

### 测试状态
- pytest 通过率: {test_report['pass_rate']}%
- 云端多模型Judge 加权均分: {test_report['avg_score']}/5.0 (GPT-4.1/Qwen-Max/DeepSeek-V3)

### 明日计划
{stats.next_tasks_preview()}

---
> 自动生成 by Orchestrator | 无需回复"""

        await self._send_dingtalk(markdown, title="📊 每日开发摘要")

        # 保存到文件
        self._save_report(f"daily/{datetime.now():%Y%m%d}.md", markdown)
        self.daily_stats.reset()

    # ─── Level 3: Sprint 报告 ───

    async def sprint_summary(self, sprint_id: int):
        """Sprint 结束时生成完整报告"""
        markdown = f"""## 🏁 Sprint {sprint_id} 完成报告

### 里程碑达成
{await self._check_milestones(sprint_id)}

### 全量统计
| 指标 | 数值 |
|------|------|
| 总任务数 | {self._total_tasks(sprint_id)} |
| 一次通过率 | {self._first_pass_rate(sprint_id)}% |
| 平均重试次数 | {self._avg_retries(sprint_id)} |
| 人工介入次数 | {self._escalation_count(sprint_id)} |

### 代码质量
- 云端多模型Judge 加权均分: {self._final_judge_score()}/5.0 (GPT-4.1/Qwen-Max/DeepSeek-V3)
```

```
- pytest 通过率: {self._final_test_pass_rate()}%
- 代码总行数: {self._total_loc()}

### 遗留问题
{self._open_issues(sprint_id)}

---
> Sprint {sprint_id} tag: v0.{sprint_id // 2 + 1}.0"""

        await self._send_dingtalk(markdown, title=f"🏁 Sprint {sprint_id} 报告")
        self._save_report(f"sprint/sprint_{sprint_id}.md", markdown)

    # ─── 异常通知 ───

    async def alert(self, level: str, message: str, context: dict = None):
        """异常告警通知"""
        emoji = {"critical": "🔴", "warning": "🟡", "info": "🟢"}[level]
        markdown = f"""## {emoji} 流水线告警

**级别**: {level.upper()}
**时间**: {datetime.now():%Y-%m-%d %H:%M:%S}
**内容**: {message}

{self._format_context(context) if context else ''}

---
> 回复「继续」恢复流水线 | 回复「暂停」暂停所有任务"""

        await self._send_dingtalk(markdown, title=f"{emoji} 告警")

    async def _send_dingtalk(self, markdown: str, title: str):
        """发送钉钉消息"""
        async with httpx.AsyncClient() as client:
            await client.post(self.webhook, json={
                "msgtype": "markdown",
                "markdown": {"title": title, "text": markdown}
            })
```

6.3 异常自愈策略

```
                    异常自愈策略矩阵
                    ═══════════════

┌─ 级别 1: 自动恢复（不通知人工）──────────────────────────┐
│                                                          │
│  SSH 连接超时（单次）                                    │
│  → 等待 30s → 重试 → 通常自动恢复                        │
│                                                          │
│  aider 编码超时（单次）                                  │
│  → kill 进程 → 拆分为更小任务 → 重新排队                 │
│                                                          │
│  git pull 冲突（rebase 可解决）                          │
│  → git pull --rebase → 自动解决 → 继续                   │
│                                                          │
│  pytest 单个 case 失败                                   │
│  → 生成修复指令 → aider 修复 → 重跑测试                  │
│                                                          │
└──────────────────────────────────────────────────────────┘

┌─ 级别 2: 自动恢复 + 通知（钉钉告警）────────────────────┐
│                                                          │
│  SSH 连接失败（连续 3 次）                               │
│  → 标记机器 DOWN → 该机器任务暂挂                        │
│  → 其他机器继续 → 钉钉通知                               │
│                                                          │
│  编码任务 3 次 Review 失败                               │
│  → 标记任务 ESCALATED → 跳过继续下个任务                 │
│  → 钉钉发送失败详情                                      │
│                                                          │
│  vLLM OOM（显存不足）                                    │
│  → 自动重启 vLLM（超时等待 3 分钟加载）                  │
│  → 如果 OOM 持续: 降低 max_model_len → 重启              │
│  → 钉钉通知                                              │
│                                                          │
│  云端多模型Judge 连续 3 轮加权评分下降                   │
│  → 暂停新任务 → 分析趋势 → 通知人工检查设计文档          │
│                                                          │
└──────────────────────────────────────────────────────────┘

┌─ 级别 3: 暂停 + 等待人工 ────────────────────────────────┐
│                                                          │
```

```
│  Git 冲突 (rebase 无法解决)                                  │
│  → 全部暂停 → 钉钉发送冲突文件 → 等待人工解决                 │
│                                                              │
│  vLLM 无法恢复 (硬件故障)                                    │
│  → 全部编码任务暂停                                          │
│  → 待硬件恢复后重启 vLLM A                                   │
│  → 钉钉告警 + 等待物理排查                                   │
│                                                              │
│  Sprint 里程碑未达标                                         │
│  → 生成详细偏差报告                                          │
│  → 暂停下一 Sprint → 等待人工决策                            │
│  → 选项: 延期 / 砍功能 / 调整设计                            │
│                                                              │
│  设计文档与代码偏差 > 30%                                    │
│  → 全面暂停 → 生成偏差分析 → 等待你更新设计文档              │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

6.4 Orchestrator 健康检查

Orchestrator 自身也需要被监控，防止调度脚本挂掉导致整条流水线停止：

```bash
#!/bin/bash
# scripts/orchestrator_watchdog.sh — Orchestrator 看门狗
# 部署为 systemd timer，每 5 分钟检查一次

ORCH_PID_FILE="/var/run/orchestrator.pid"
ORCH_HEALTH="http://localhost:9500/health"
DINGTALK_WEBHOOK="${DINGTALK_WEBHOOK_URL}"

# 检查 Orchestrator 是否存活
if ! curl -sf "$ORCH_HEALTH" > /dev/null 2>&1; then
    echo "$(date) Orchestrator 不可达，尝试重启..."

    # 检查进程是否存在
    if [ -f "$ORCH_PID_FILE" ]; then
        kill -9 $(cat "$ORCH_PID_FILE") 2>/dev/null
    fi

    # 重启
    cd ~/ai-crm
    nohup python -m orchestrator.main > logs/orchestrator.log 2>&1 &
    echo $! > "$ORCH_PID_FILE"

    sleep 10

    # 验证重启成功
    if curl -sf "$ORCH_HEALTH" > /dev/null 2>&1; then
        echo "$(date) Orchestrator 重启成功"
        # 通知钉钉
        curl -s "$DINGTALK_WEBHOOK" \
            -H 'Content-Type: application/json' \
            -d '{"msgtype": "text", "text": {"content": "🟡 Orchestrator 自动重启成功，流水
    else
        echo "$(date) Orchestrator 重启失败"
        curl -s "$DINGTALK_WEBHOOK" \
            -H 'Content-Type: application/json' \
            -d '{"msgtype": "text", "text": {"content": "🔴 Orchestrator 重启失败，需要人工
    fi
fi
```

systemd 配置：

```
# /etc/systemd/system/orchestrator.service
[Unit]
Description=AI CRM Orchestrator
After=network.target

[Service]
Type=simple
User=realman
WorkingDirectory=/home/realman/ai-crm
ExecStart=/usr/bin/python -m orchestrator.main
Restart=on-failure
RestartSec=30
PIDFile=/var/run/orchestrator.pid

[Install]
WantedBy=multi-user.target

# /etc/systemd/system/orchestrator-watchdog.timer
[Unit]
Description=Orchestrator Watchdog Timer

[Timer]
OnBootSec=1min
OnUnitActiveSec=5min

[Install]
WantedBy=timers.target
```

6.5 Orchestrator Dashboard API

Orchestrator 暴露 HTTP API，供查看实时状态：

```python
# orchestrator/dashboard.py — 状态查询 API

from fastapi import FastAPI

app = FastAPI(title="Orchestrator Dashboard", port=9500)

@app.get("/health")
async def health():
    return {"status": "running", "uptime": get_uptime()}

@app.get("/api/status")
async def status():
    """当前流水线状态"""
    return {
        "current_sprint": engine.current_sprint,
        "tasks": {
            "total": engine.total_tasks,
            "completed": engine.completed_count,
            "in_progress": engine.in_progress_count,
            "failed": engine.failed_count,
            "queued": engine.queued_count,
        },
        "machines": {
            machine: {
                "status": dispatcher.machine_status(machine),
                "current_task": dispatcher.current_task(machine),
                "completed_today": engine.machine_completed_today(machine),
            }
            for machine in ["4090", "mac_min_8T", "gateway", "data_center"]
        },
        "last_test_report": reporter.last_test_report(),
        "next_batch": [t.task_id for t in engine.next_batch()],
    }

@app.get("/api/tasks/{task_id}")
async def task_detail(task_id: str):
    """查看单个任务的详细状态"""
    return engine.get_task(task_id)

@app.post("/api/pause")
async def pause():
    """暂停流水线"""
    engine.pause()
```

```python
    return {"status": "paused"}

@app.post("/api/resume")
async def resume():
    """恢复流水线"""
    engine.resume()
    return {"status": "running"}

@app.post("/api/docs-changed")
async def docs_changed(files: list[str]):
    """接收文档变更通知（post-receive hook 调用）"""
    engine.trigger_reparse(files)
    return {"status": "reparse_triggered"}
```

6.6 全自动一天的运行时序

```
                    全自动一天的运行时序
                    ═══════════════════

00:00  Orchestrator 持续运行中...
       自动执行夜间低优先级任务（代码重构、文档生成）

02:00  data_center 自动触发 PG 备份（Celery 定时任务）
       → 备份完成通知写入日志

08:00  Orchestrator 切入日间模式
       ├── 从 Redis 取出今日的任务队列
       ├── 检查 5 台机器状态
       ├── 开始分发编码任务
       └── 钉钉发送："🟢 今日开发启动，共 XX 个任务"

08:05  第 1 批（4 个任务并行）
~      ├── 4090:       aider 编码 agent/supervisor.py
08:20  ├── mac_min_8T: aider 编码 crm/api/leads.py
       ├── gateway:    aider 编码 deploy/nginx/nginx.conf
       └── data_center: aider 编码 deploy/monitoring/prometheus.yml

08:20  自动 Review（三层）
~      ├── 3 个通过，1 个需要修复
08:25  └── 自动生成修复指令，gateway 重新编码

08:25  git push + orchestrator 本地自动测试触发
~      ├── orchestrator: pytest → 云端API多模型Judge（本地执行，零延迟）
08:30  └── 报告生成：12/15 case 通过

08:35  第 2 批（4 个新任务）
       └── 同上循环...

...    （全天持续自动循环）

12:00  午间检查点
       ├── Orchestrator 自动检查上午进度
       └── 如果进度落后 → 调整任务优先级

...    （下午继续自动循环）

18:00  每日摘要报告
```

```
        ├── 钉钉发送每日开发摘要 (Level 2 报告)
        ├── 保存日报到 reports/daily/
        └── 你查看手机钉钉即可了解今日进展

18:05  夜间模式
        ├── 继续执行低优先级任务
        ├── 代码重构、测试补全
        └── 如果任务队列清空 → IDLE 等待
```

完全不需要人工介入。你只在以下情况收到通知:

- 有任务升级需要人工处理
- 每日 18:00 摘要
- Sprint 结束报告